

Universidad del Valle

Facultad de Ingeniería

Escuela de Ingeniería de Sistemas y Computación

Inteligencia Artificial

Proyecto 1

Robotaxi Zoox

Estudiantes:

Nombre del estudiante 1

Nombre del estudiante 2

Nombre del estudiante 3

Profesor:

Nombre del profesor

Semestre:

2026-I

29 de mayo de 2026

Índice

1. Descripción del problema	2
2. Resumen del proyecto	2
3. Modelado del problema	2
3.1. Representación del estado	2
3.2. Operadores	3
3.3. Función de costo	3
3.4. Condición de meta	3
4. Algoritmos de búsqueda no informada	4
4.1. Búsqueda en amplitud	4
4.2. Costo uniforme	4
4.3. Profundidad evitando ciclos	4
4.4. Cómo se evitan los ciclos en no informada	4
5. Algoritmos de búsqueda informada	5
5.1. Heurística utilizada	5
5.2. Búsqueda avara	5
5.3. Búsqueda A*	6
5.4. Admisibilidad de la heurística	6
5.4.1. Caso 1: aún faltan pasajeros	6
5.4.2. Caso 2: todos los pasajeros ya fueron recogidos	6
5.4.3. Conclusión	7
5.5. Cómo se evitan los ciclos en avara y A*	7
5.5.1. Avara	7
5.5.2. A*	7
5.6. Precisiones sobre los algoritmos no completos	7
6. Interfaz y ejecución	8
7. Conclusiones	8

1. Descripción del problema

El proyecto plantea una versión avanzada del *Robotaxi Zoox*, capaz de recoger múltiples pasajeros dentro de una ciudad modelada como una cuadrícula de 10×10 , para finalmente llevarlos a un único destino. Cada casilla del ambiente representa un tipo de elemento del mundo: vía libre, muro, punto de inicio, flujo vehicular alto, pasajero o destino.

El vehículo solo puede desplazarse en las cuatro direcciones cardinales: arriba, abajo, izquierda y derecha. El costo del desplazamiento depende del tipo de casilla a la que se llega:

- Vía libre, inicio, pasajero y destino tienen costo 1.
- Flujo vehicular alto tiene costo 7.

Adicionalmente, si el robot entra a una casilla con pasajero, este se recoge automáticamente. El objetivo de búsqueda consiste en encontrar una secuencia de movimientos que recoja a todos los pasajeros y termine en el destino, respetando las restricciones del mapa y minimizando el criterio correspondiente según el algoritmo aplicado.

2. Resumen del proyecto

En este proyecto se desarrolló una aplicación interactiva que:

- carga un mundo desde un archivo de texto;
- representa gráficamente el mapa de la ciudad;
- permite elegir algoritmos de búsqueda no informada e informada;
- anima visualmente el recorrido encontrado;
- muestra un reporte con profundidad, nodos expandidos, tiempo de cómputo, costo y, para la búsqueda informada, el valor heurístico del nodo solución.

La implementación actual organiza la lógica en módulos separados para el mundo, las búsquedas, la ejecución y la interfaz gráfica. Esta separación facilita la trazabilidad entre el modelo del problema, la estrategia de búsqueda y la visualización final.

3. Modelado del problema

3.1. Representación del estado

Cada nodo del espacio de búsqueda se modela por medio de:

- la posición actual del vehículo, representada como una tupla (*fila*, *columna*);

- el conjunto de pasajeros recogidos hasta el momento;
- el costo acumulado g ;
- la heurística h , cuando el algoritmo la requiere;
- un puntero al padre, para reconstruir el camino solución;
- la profundidad del nodo.

La decisión de incluir el conjunto de pasajeros recogidos dentro del estado es esencial. Gracias a ello, dos nodos en la misma posición pueden considerarse distintos si difieren en los pasajeros ya recogidos. Esta decisión permite cumplir con el lineamiento del proyecto de **evitar ciclos**, pero al mismo tiempo **permitir devolverse** cuando el vehículo ya cambió su estado al recoger un pasajero.

3.2. Operadores

La expansión de nodos se realiza con cuatro movimientos posibles:

$$(0, 1), (-1, 0), (1, 0), (0, -1)$$

que corresponden a derecha, arriba, abajo e izquierda, respectivamente. Solo se generan vecinos transitables; es decir, casillas válidas dentro del mapa y diferentes de muro.

3.3. Función de costo

La función de costo distingue entre dos tipos de tránsito:

- flujo bajo: costo 1;
- flujo alto: costo 7.

Por tanto, el costo acumulado de un nodo es la suma de los costos de las casillas a las que el vehículo ha ido entrando desde el estado inicial.

3.4. Condición de meta

Un nodo se considera meta si y solo si:

1. la posición del vehículo coincide con el destino, y
2. el conjunto de pasajeros recogidos coincide con el conjunto total de pasajeros del mapa.

4. Algoritmos de búsqueda no informada

La implementación unifica amplitud, costo uniforme y profundidad evitando ciclos dentro de un mismo envoltorio en `algoritmosBusqueda/noinformada/wrapper.py`. La diferencia entre algoritmos está en la estructura de frontera y la forma de extraer el siguiente nodo.

4.1. Búsqueda en amplitud

La búsqueda en amplitud (**bfs**) usa una cola FIFO. Esto hace que primero se exploren los nodos de menor profundidad. Por construcción, encuentra soluciones con el menor número de pasos cuando todos los costos son uniformes.

4.2. Costo uniforme

La búsqueda de costo uniforme (**ucs**) usa una cola de prioridad ordenada por el costo acumulado g . De este modo, el siguiente nodo a expandir siempre es el de menor costo total conocido hasta el momento.

4.3. Profundidad evitando ciclos

La búsqueda en profundidad (**dfs**) usa una pila LIFO. Como la estrategia profundiza rápidamente en una rama, se vuelve especialmente importante controlar los ciclos para evitar trayectorias infinitas o expansiones redundantes.

4.4. Cómo se evitan los ciclos en no informada

El control de ciclos se hace sobre el estado compuesto (*posicion, pasajeros_recogidos*). Esto significa que solo se descarta un nodo si vuelve exactamente a una posición con el mismo conjunto de pasajeros recogidos.

Listing 1: Control de ciclos en búsqueda no informada

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)

if estado in visitados:
    continue

visitados.add(estado)
```

Esta decisión es particularmente importante en **dfs**. Si el vehículo vuelve a una casilla ya visitada pero ahora con más pasajeros recogidos, el estado ya no es el mismo y por tanto la expansión sí se permite. Esto implementa directamente la regla del proyecto: **evitar ciclos, pero dejar que el vehículo se devuelva después de recoger un pasajero**.

5. Algoritmos de búsqueda informada

La búsqueda informada se implementa en `algoritmosBusqueda/informada/wrapper.py`. Allí se soportan dos variantes: búsqueda avara y búsqueda A*.

5.1. Heurística utilizada

La heurística base es la distancia Manhattan:

$$h_M((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2|$$

Sobre esa base, el proyecto define la función heurística de esta forma:

- si el vehículo ya recogió a todos los pasajeros, la heurística es la distancia Manhattan desde la posición actual hasta el destino;
- si todavía faltan pasajeros, la heurística es la distancia Manhattan al pasajero faltante más cercano.

En términos operativos:

Listing 2: Heurística implementada

```
def calcular_heuristica(nodo, destino, pasajeros, meta_pasajeros):
    :
    if nodo.pasajeros_recogidos == meta_pasajeros:
        return heuristica_manhattan(nodo.posicion, destino)

    pasajeros_faltantes = [p for p in pasajeros if p not in nodo.
                           pasajeros_recogidos]
    return min(heuristica_manhattan(nodo.posicion, p) for p in
               pasajeros_faltantes)
```

5.2. Búsqueda avara

La búsqueda avara prioriza únicamente la heurística h . Es decir, siempre expande primero el nodo que parece más cercano a una meta intermedia según la estimación heurística.

$$f(n) = h(n)$$

Esta estrategia puede ser rápida en algunos casos, pero no garantiza optimalidad porque ignora el costo acumulado g .

5.3. Búsqueda A*

La búsqueda A* combina el costo acumulado con la estimación heurística:

$$f(n) = g(n) + h(n)$$

Esto le permite balancear el costo ya pagado y el costo estimado faltante. Dado que la heurística utilizada es admisible, A* mantiene la garantía estándar de encontrar una solución óptima respecto al costo.

5.4. Admisibilidad de la heurística

La heurística propuesta es admisible porque nunca sobreestima el costo real restante hasta la meta. La justificación depende del caso.

5.4.1. Caso 1: aún faltan pasajeros

Si todavía falta al menos un pasajero por recoger, cualquier solución válida debe llegar primero a **algún** pasajero faltante antes de poder terminar en el destino. La heurística toma la distancia Manhattan al pasajero faltante más cercano.

La distancia Manhattan subestima o iguala el número de movimientos necesarios en una cuadrícula con movimientos ortogonales, porque:

- ignora obstáculos;
- ignora rodeos;
- supone implícitamente que cada movimiento cuesta al menos 1.

Además, en este proyecto el costo real de un movimiento nunca es menor que 1, y algunas casillas cuestan 7. Por lo tanto:

$$h(n) \leq \text{costo real para alcanzar al menos un pasajero faltante} \leq \text{costo real restante hasta la meta}$$

5.4.2. Caso 2: todos los pasajeros ya fueron recogidos

En este caso la tarea restante se reduce a ir desde la posición actual hasta el destino. De nuevo, la distancia Manhattan al destino nunca sobreestima el costo real, porque cada desplazamiento vale al menos 1 y la presencia de muros o tráfico alto solo puede aumentar el costo real.

$$h(n) = h_M(n, \text{destino}) \leq h^*(n)$$

donde $h^*(n)$ representa el costo óptimo real desde n hasta la meta.

5.4.3. Conclusión

En ambos casos:

$$h(n) \leq h^*(n)$$

por lo que la heurística es admisible y satisface lo solicitado por el enunciado del proyecto.

5.5. Cómo se evitan los ciclos en avara y A*

5.5.1. Avara

En búsqueda avara se usa un conjunto `visitados` sobre el estado compuesto (*posicion, pasajeros_recogidos*)

Listing 3: Control de ciclos en Avara

```
estado = (nodo.posicion, nodo.pasajeros_recogidos)

if estado in visitados:
    continue
visitados.add(estado)
```

Esto evita reexpandir el mismo estado indefinidamente. Sin embargo, si el vehículo regresa a una posición después de haber recogido un pasajero, el conjunto `pasajeros_recogidos` cambia y el estado es distinto. En consecuencia, el retorno sí se permite.

5.5.2. A*

En A* el control es más fino: no solo se evita repetir estados, sino que además se conserva el mejor costo conocido para cada estado.

Listing 4: Control de repetición de estados en A*

```
if estado in best_g and best_g[estado] <= nodo.g:
    continue
best_g[estado] = nodo.g
```

Así, un estado solo se descarta si ya fue alcanzado antes con un costo menor o igual. Si se encuentra el mismo estado con mejor costo, se conserva la nueva alternativa. Este mecanismo evita ciclos costosos y mantiene la lógica correcta de A*.

5.6. Precisiones sobre los algoritmos no completos

Dentro de la práctica del proyecto, los casos más delicados frente a ciclos son `dfs` y `avara`. En ambos algoritmos la estrategia de expansión puede quedarse atrapada explorando repeticiones si no existe control explícito de estados. Por eso, en esta implementación:

- `dfs` evita ciclos mediante el conjunto `visitados` basado en (*posicion, pasajeros_recogidos*);

- **avara** usa exactamente el mismo criterio de estado para no reexpandir estados ya explorados;
- en ambos casos, el retorno a una casilla previamente recorrida sí se permite cuando el conjunto de pasajeros recogidos ya cambió.

Esta última propiedad es la que hace compatible la prevención de ciclos con el requisito del enunciado.

6. Interfaz y ejecución

La aplicación carga un archivo de texto, dibuja el mapa, permite escoger la familia de búsqueda y luego el algoritmo concreto desde la interfaz gráfica. Posteriormente anima el recorrido y muestra un reporte con:

- nodos expandidos;
- profundidad;
- tiempo de cómputo;
- costo, cuando aplica;
- heurística final, en el caso de la búsqueda informada.

La versión actual del proyecto fue reorganizada para dejar una interfaz persistente: el usuario puede mantener el mismo mapa en pantalla, probar diferentes algoritmos y cambiar de mapa desde la misma sesión sin cerrar el programa.

7. Conclusiones

El proyecto permite contrastar claramente el comportamiento de búsquedas no informadas e informadas sobre un mismo ambiente con costos variables. La principal ventaja de la modelación adoptada es que el estado incluye tanto la posición como los pasajeros ya recogidos, lo que resuelve correctamente el requisito de evitar ciclos sin impedir retornos necesarios después de recoger pasajeros.

La heurística basada en distancia Manhattan al pasajero faltante más cercano, o al destino cuando ya no faltan pasajeros, es simple, eficiente y admisible bajo las reglas del problema. Esto hace que A* sea una alternativa sólida para encontrar soluciones de bajo costo dentro de la ciudad modelada.

Nota de edición: antes de entregar, reemplace los campos de estudiantes y profesor en la portada por los nombres reales del grupo.